

What is Transfer Learning and Finetuning?

Monday, April 28, 2025 4:04 PM

High level feature will be same (balance, brake, horn etc) hence here we can apply the transfer learning whereas, for low level feature (gearbox, clutching system) can apply fine tuning.

- "Transfer learning means taking experience from one problem and using it to solve another related problem."

Real Life Example: "It's like learning to ride a bicycle and then using that skill to learn how to ride a motorcycle faster."

Computer Scientist took this simple idea and integrated into the deep learning.

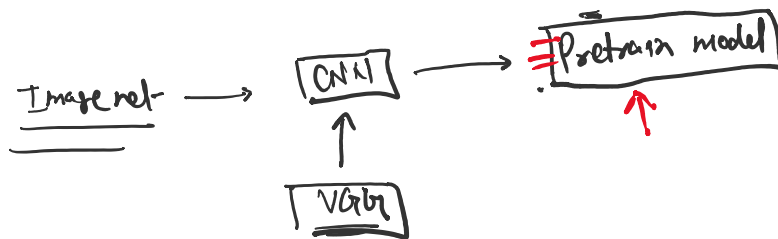
ILSVRC (ImageNet Large Scale Visual Recognition Challenge) version mein:

- Training Images: ~1.2 million
- Validation Images: 50,000
- Test Images: 100,000 (labels hidden, competition ke liye)
- Classes: 1000 (dog, cat, car, airplane, etc.)
- Full ImageNet Dataset (original, jo Stanford maintain karta hai):
 - 14+ million images
 - 21,841 categories (WordNet synsets ke hisaab se)

This is example of those 1000 classes

Cat
Dog
Vehicle
Aircraft
Bird
Fruit
Vehicle
Sports Object
Musical Instrument
Humans

ImageNet



Pre trained model can identify whether image is of dog or no dog but will fail to identify specific breed of dog like Golden Retriever. Since high level feature for identifying the dog will remain same (Tail, fur, eyes nose etc) but low level feature will change (To identify Golden Retriever) therefore, we can fine tune the Pre Trained model that has already captured the high level features of Dog using training data images of Golden Retriever capturing the low level feature of the same

Now suppose your specific class is not included among the 1000 classes available in the ImageNet dataset.



Now the ways of doing this Transfer Learning:

In CNN based Architecture:

- Replacing the output layer of pretrain model
- Or don't change weights means keep freezing all the weights just retrain the Neural Network Weights
- Or unfreezing some layers of the network and fine-tuning it on the new, task-specific dataset

Now Lets take a very simple example:

First understand it using the CNN based Architecture.

VGG16 MODEL ARCHITECTURE

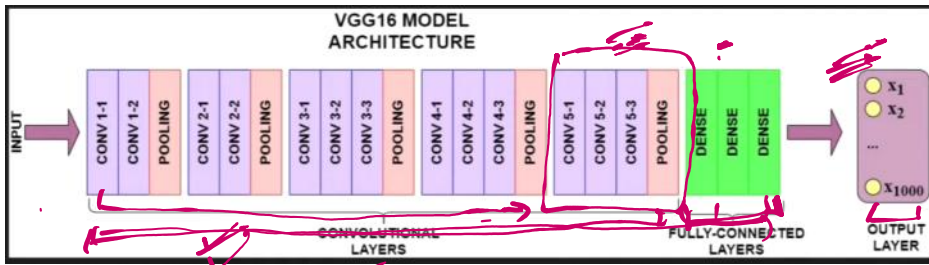
In below VGG archⁿ train the Dense layer highlighted with Green

- Unfreezing the couple of last Convolution Layers.
- Why only last layers and not the starting ones?
- Because, Starting ones are responsible for capturing the high level features, whereas last layers captures the low level features which we want to fine tune in case we wish to use the pre trained model in the areas whereas we can harness the power of this super intelligent model

Rahul Suman

In simple reusing the Pre Trained model is Transfer Learning But, Doing some tweaking in the Pre Trained model so that I can perform the new task is called Fine Tuning

First understand it using the CNN based Architecture.



Part of the Pre Trained model that we tweak is considered as Fine Tuning, Whereas part of the Pre Trained model that is freeze is considered as Transfer learning

Label output

Pre =

Note car eyes lips

Why this transfer learning works

We train a model on a **huge dataset** initially, allowing it to learn **primitive features** (like edges, textures, and simple shapes). When applying it to a **different dataset**, we don't retrain the entire model — we only retrain certain layers if the new data is **very different**. Otherwise, we can directly reuse the model's knowledge.

= FC/FT

Act Pin

L. binary → Sigmoid
L. multi → Softmax

Specific

Note car eyes

(Pretrain) → (Transfer Learning) = FT Model

Transfer Learning in Generative AI (Using LLMs)

In Transformer based Architecture:

- Replacing the **output layer** of pretrain model (As discussed in CNN fine tuning case)
- Or **unfreezing some layers** of the network and **fine-tuning** it on the new, task-specific dataset (Nx as discussed below)

Large PEFT GPT 3.5, 4, 4V

Transformer

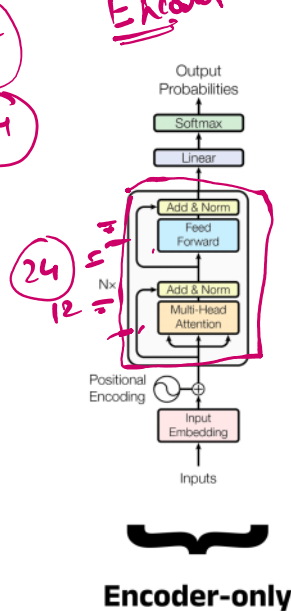
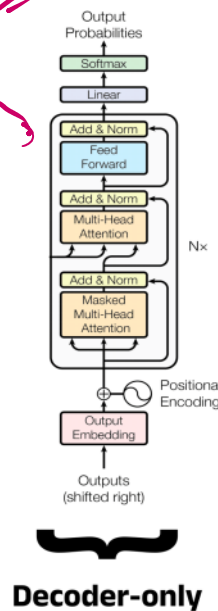
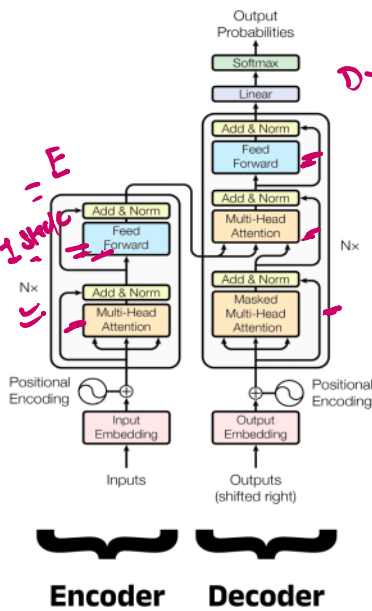
GPT*

BERT*

12 24

In the Architectures alongside Nx means N times x, where x represents the outer box as single entity which is repeated N number of times.

Therefore each x represents a layer and in Fine Tuning we can tweak some of the last x in the architecture



*Illustrative example, exact model architecture may vary slightly

CNN Finetuning

Wednesday, April 23, 2025 6:54 PM

Correct: <https://colab.research.google.com/drive/1Dtipjnr4WXnl8ErDeNzzOjzGtgmV-F4r?usp=sharing>

<https://colab.research.google.com/drive/1U57ZndGQ6WnPdtM2rTdxPARNnTsfRds7?usp=sharing>

Bert Finetuning

Monday, April 28, 2025 4:04 PM

<https://colab.research.google.com/drive/1Dtipjnr4WXnl8ErDeNzzOjzGtgmV-F4r?usp=sharing>

<https://colab.research.google.com/drive/1U57ZndGQ6WnPdtM2rTdxPARNnTsfRds7?usp=sharing>

`include_top = True`

Google Colab, Kaggle Kernel, Paper Space

Pooling Layer having 0 trainable parameters hence it is non trainable

`lable = 'inferred'`